

AI Gaming / Team Cyber Tesla / NeuroBridge.SI Baku 2026

PlayerOne

An AI playtester. It plays your game, catches the bug, and files the report with proof.

A free local AI does the playing. **A big AI only does the judging.**

`roaring-alfajores-8a941a.netlify.app`
`github.com/nijathatamli/AI-hakaton`

Small studios ship bugs because nobody has time to replay the game

Manual testing

Slow and repetitive. Nobody replays level 3 for the hundredth time after every build.

Hiring QA

Too expensive for a team of 1 to 15, and still slow to turn around.

A big AI playing

Reads a full screenshot every move: 67,000 to 85,000 tokens per minute of play.

Split the brain.

Cheap hands, smart judge.

1

Play

free, on
your PC

A local vision AI or our free bot plays with real keys and mouse. Any engine, no SDK.

2

Watch

exact,
zero
tokens

Reads the engine console, catches crashes, hangs and soft-locks, keeps the last 12 seconds.

3

Judge

your AI,
tiny
digests

Claude, Codex or Gemini read a 45 KB summary, decide what is a real bug and write the ticket.

The prototype

Pick a game. It knows how to play.

PlayerOne recognises the game, suggests controls, and you approve exactly what the AI may press. It never touches Alt+F4. F8 stops it anywhere.

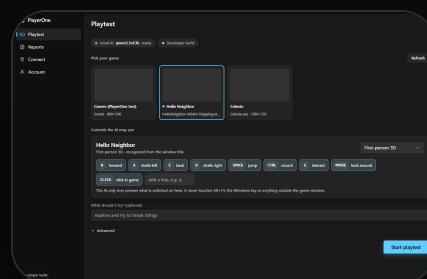
v0.2.3

released,
with
installers

5

targets:
Windows

x64



The desktop app: pick a window, approve controls, press Start.

what it does, and what it does not

Plays from pixels

A vision model reads each frame and picks keys, mouse look and clicks from the approved controls. Stuck? It turns away and tries something else.

Works out the controls

Recognises the game by name or from the screen and picks a layout: first-person, platformer, top-down.

Judges with evidence

The big AI gets console errors and a contact sheet of the seconds before

Detection is rules, on purpose

Crash, hang, console error and soft-lock checks need no

What a report looks like

Proof, not "it broke somewhere"

Error and stack trace

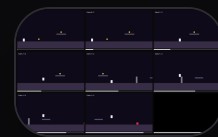
```
res://main.gd:154
```

Repro steps

the recorded inputs that led there

Contact sheet and clip

36 KB picture, 14 KB 480p clip



Contact sheet: the 9 frames before the crash.

Five planted bugs plus a clean build. One minute of real play each, three players, 18 runs. Every log, frame and clip is in the repo.

Bug	Random keys	Free bot	What broke, and what we did PlayerOne froze with a frozen game. Capture now has a 1.5 s deadline. 13 false alarms. It now tries every control before calling it stuck. Down to 0. 8 unit tests now cover
Soft-lock	34 s	15 s	
Script error	missed	12.5 s	
Crash	missed	18 s	
Freeze	missed	20 s	
Coin counter	missed	missed	

The same minute of testing, 96% fewer tokens

70,026 2,975

tokens a big AI spends
playing one minute itself

about \$0.14 per minute

tokens it spent judging
that minute through
PlayerOne

about \$0.006 per minute

The local AI is slow

not.
25 to
45
moves
a
minute
against
77 for
the bot
The bot
is the
better
driver;
the AI is
better
at
judging

A weak judge writes weak reports

The 3B
judge
wrote
"press
key".
PlayerC
now fa
back to

time, code or tokens

Approach	Examples	Setup	Exact engine errors	Runs after every build
Manual QA	the dev, friends, outsourced QA studios	none	by hand	no
Scripted test automation	GameDriver, Unity Test Framework, Godot GUT	SDK and written tests	yes	yes
AI game-testing platforms	modl.ai and similar	engine integration	yes	yes
Computer-use agents	Claude, OpenAI computer use	none	no	too costly

Who we sell to

Indie and small studios of 1 to 15 people shipping PC games in Godot, Unity and Unreal. Thousands of new PC games reach Steam every year, most from small teams.

How we reach them

Godot and Unity communities, itch.io and game jams, then Baku

Plan

Who pays
for what

Our cost
to serve

Free

Local AI, crash and console capture

\$0, runs on their PC

Connect

Monthly. Their own Claude, Codex or Gemini key judges

near \$0, they pay the model

Studio

Per seat. Shared reports, a playtest in CI after every build

storage and sync

Feasibility

Runs today, on hardware people already own

Data
needed

None.
No training data and no game source code. It works from the screen and the engine console.

Running
cost

Free on the user's PC. With a cloud judge, about \$0.36 per hour of play instead of \$8.40.

Already
live

Accounts, plans and usage metering on Supabase. Stripe webhook. Installers for 5 targets.

engine plugin. **Any game on screen.**

Other tools need engine integration or run the expensive model on every frame. PlayerOne attaches to any window, trusts the engine console first, and only sends the big model a compressed digest.

Try it

Next 30 days

**Pilot
with
three
Baku
indie
studios
on their
real
games.
Measure
bugs
found
per
build.**

Then: Unity
and Unreal
log probes
on real